

Integrációs tesztek - Dokumentáció

April 30, 2026

1 integration.dashboard package

1.1 Submodules

1.2 integration.dashboard.test_auth module

1.2.1 test_dashboard_login_rejects_wrong_password(dashboard_base_url: str, administrator_email: str, dashboard_request_headers: dict[str, str])

Verify login fails with a clear error when an incorrect password is provided.

1.2.2 test_dashboard_login_returns_bearer_token(dashboard_base_url: str, administrator_email: str, administrator_password: str, dashboard_request_headers: dict[str, str])

Verify dashboard login returns a usable bearer access token for admin credentials.

1.2.3 test_dashboard_me_returns_admin_user(dashboard_base_url: str, admin_token: str, administrator_email: str, dashboard_request_headers: dict[str, str])

Verify the authenticated /users/me endpoint returns administrator identity fields.

1.3 integration.dashboard.test_gateway_health module

1.3.1 test_dashboard_health_endpoint_returns_200(dashboard_base_url: str, dashboard_request_headers: dict[str, str], frontend_origin: str)

Verify dashboard health endpoint returns 200 and expected CORS headers.

1.4 integration.dashboard.test_keys_and_quotas module

1.4.1 test_keys_and_quotas_key_access_then_quota_limit_enforced(`dashboard_base_url: str`, `completions_url: str`, `admin_token: str`, `dashboard_request_headers: dict[str, str]`, `completions_dir: Path`)

Verify a created key works first, then a strict quota eventually blocks further requests.

1.5 integration.dashboard.test_projects module

1.5.1 test_projects_add_user_to_project(`dashboard_base_url: str`, `admin_token: str`, `dashboard_request_headers: dict[str, str]`)

Verify adding a user to a project updates the project's user membership view.

1.5.2 test_projects_create_project_then_list_contains_it(`dashboard_base_url: str`, `admin_token: str`, `dashboard_request_headers: dict[str, str]`)

Verify creating a project makes it retrievable from subsequent list calls.

1.5.3 test_projects_delete_project_then_second_delete_fails(`dashboard_base_url: str`, `admin_token: str`, `dashboard_request_headers: dict[str, str]`)

Verify deleting a project succeeds once and then fails on repeated deletion.

1.5.4 test_projects_remove_user_from_project(`dashboard_base_url: str`, `admin_token: str`, `dashboard_request_headers: dict[str, str]`)

Verify removing a user from a project removes them from project membership results.

1.5.5 test_projects_seeded_projects_are_listed(`dashboard_base_url: str`, `admin_token: str`, `dashboard_request_headers: dict[str, str]`)

Verify seeded projects are visible through the dashboard project listing endpoint.

1.6 integration.dashboard.test_users module

1.6.1 test_users_admin_can_change_password_for_user(`dashboard_base_url: str, admin_token: str, dashboard_request_headers: dict[str, str]`)

Verify admin password change invalidates old password and allows login with the new one.

1.6.2 test_users_admin_can_delete_user(`dashboard_base_url: str, admin_token: str, dashboard_request_headers: dict[str, str]`)

Verify admin can delete a user and receives not-found on repeated delete.

1.6.3 test_users_me_shows_new_user_and_password_expired_when_mandated(`dashboard_base_url: str, admin_token: str, dashboard_request_headers: dict[str, str]`)

Verify a mandated-reset user can authenticate and sees password-expired state in `/users/me`.

1.6.4 test_users_register_invalid_email_returns_verbose_422_detail(`dashboard_base_url: str, admin_token: str, dashboard_request_headers: dict[str, str]`)

Verify invalid registration payload returns readable validation detail text.

1.6.5 test_users_register_with_mandated_reset(`dashboard_base_url: str, admin_token: str, dashboard_request_headers: dict[str, str]`)

Verify admin registration creates a user and mandates password reset when requested.

2 integration.gateway package

2.1 Submodules

2.2 integration.gateway.test_chat_end_to_end module

2.2.1 test_chat_end_to_end(`completions_url: str, unlimited_api_key: str, completions_dir: Path`)

Verify a valid chat completion request returns the expected mock completion payload.

2.3 integration.gateway.test_dashboard_health module

2.3.1 test_health_endpoint_returns_200(base_url: str)

Verify the gateway health endpoint reports an operational status.

2.4 integration.gateway.test_key_authentication module

2.4.1 test_key_authentication_rejects_expired_key(completions_url: str, expired_api_key: str, completions_dir: Path)

Ensure expired API keys cannot access completions.

2.4.2 test_key_authentication_rejects_invalid_fingerprint_with_valid_length(completions_url: str, completions_dir: Path)

Ensure valid-length but unknown key fingerprints are rejected.

2.4.3 test_key_authentication_rejects_invalid_key_with_matching_fingerprint(completions_url: str, completions_dir: Path)

Ensure keys with matching fingerprint but invalid secret are rejected.

2.4.4 test_key_authentication_rejects_too_long_key(completions_url: str, completions_dir: Path)

Ensure oversized API keys are rejected as invalid fingerprints.

2.4.5 test_key_authentication_rejects_too_short_key(completions_url: str, completions_dir: Path)

Ensure malformed short API keys are rejected with a validation error.

2.5 integration.gateway.test_quota_enforcement module

2.5.1 test_quota_immediate_over_limit_returns_429(completions_url: str, limited_api_key: str, completions_dir: Path)

Verify oversized requests are immediately rejected when quota would be exceeded.

2.5.2 test_quota_within_limit_then_next_request_exceeds_remaining_quota(completions_url: str, limited_api_key: str, completions_dir: Path)

Verify one in-quota request succeeds and the following request is rate-limited.

2.6 integration.gateway.test_upstream_error module

2.6.1 test_upstream_error(completions_url: str, unlimited_api_key: str, completions_dir: Path)

Verify upstream error mappings are propagated with expected status and payload.

3 integration package

3.0.1 auth_headers(admin_token: str, dashboard_request_headers: dict[str, str]) → dict[str, str]

Merge dashboard request headers with an admin bearer authorization header.

- **Parameters:**

- **admin_token** (*str*) – Access token for an authenticated administrator.
- **dashboard_request_headers** (*dict* [***str**, *str* ***]) – Baseline request headers required by dashboard routes.

- **Returns:** A headers dictionary with the Authorization header included.

- **Return type:** *dict*[*str*, *str*]

3.0.2 bearer_headers(token: str) → dict[str, str]

Build Authorization headers for bearer token authentication.

- **Parameters:** **token** (*str*) – The bearer token value.
- **Returns:** A headers dictionary containing the Authorization header.
- **Return type:** *dict*[*str*, *str*]

3.0.3 build_chat_url(completions_url: str) → str

Build the chat completions endpoint URL from a base completions URL.

- **Parameters:** **completions_url** (*str*) – The base URL prefix for completions requests.
- **Returns:** The fully qualified chat completions endpoint URL.
- **Return type:** *str*

3.0.4 create_api_key(dashboard_base_url: str, headers: dict[str, str], project_id: int, name: str) → Response

Create an API key through the dashboard keys API.

- **Parameters:**

- **dashboard_base_url** (*str*) – Base URL of the dashboard backend service.
- **headers** (*dict* [***str**, *str* **/*]) – Authenticated request headers.
- **project_id** (*int*) – Project ID the API key will belong to.
- **name** (*str*) – Human-readable API key label.
- **Returns:** The HTTP response from the create key endpoint.
- **Return type:** requests.Response

3.0.5 create_project(dashboard_base_url: str, headers: dict[str, str], project_name: str, manager_id: int) → Response

Create a project through the dashboard projects API.

- **Parameters:**
 - **dashboard_base_url** (*str*) – Base URL of the dashboard backend service.
 - **headers** (*dict* [***str**, *str* **/*]) – Authenticated request headers.
 - **project_name** (*str*) – Name for the new project.
 - **manager_id** (*int*) – User ID assigned as project manager.
- **Returns:** The HTTP response from the create endpoint.
- **Return type:** requests.Response

3.0.6 create_quota(dashboard_base_url: str, headers: dict[str, str], key_id: int | None = None, project_id: int | None = None, user_id: int | None = None, limit_id: int, limit_value: float, period: str) → Response

Create a quota through the dashboard quotas API.

- **Parameters:**
 - **dashboard_base_url** (*str*) – Base URL of the dashboard backend service.
 - **headers** (*dict* [***str**, *str* **/*]) – Authenticated request headers.
 - **key_id** (*int* | *None*, *optional*) – Optional API key ID target for the quota.
 - **project_id** (*int* | *None*, *optional*) – Optional project ID target for the quota.
 - **user_id** (*int* | *None*, *optional*) – Optional user ID target for the quota.
 - **limit_id** (*int*) – Limit type identifier.
 - **limit_value** (*float*) – Numeric quota amount.
 - **period** (*str*) – Quota period value as API string.
- **Returns:** The HTTP response from the quota create endpoint.
- **Return type:** requests.Response

3.0.7 `delete_api_key`(`dashboard_base_url`: `str`, `headers`: `dict[str, str]`, `key_id`: `int`) → `Response`

Delete an API key through the dashboard keys API.

- **Parameters:**
 - `dashboard_base_url` (`str`) – Base URL of the dashboard backend service.
 - `headers` (`dict [**str*`, `str */`) – Authenticated request headers.
 - `key_id` (`int`) – Identifier of the API key to delete.
- **Returns:** The HTTP response from the delete key endpoint.
- **Return type:** `requests.Response`

3.0.8 `delete_project`(`dashboard_base_url`: `str`, `headers`: `dict[str, str]`, `project_id`: `int`) → `Response`

Archive or delete a project through the dashboard projects API.

- **Parameters:**
 - `dashboard_base_url` (`str`) – Base URL of the dashboard backend service.
 - `headers` (`dict [**str*`, `str */`) – Authenticated request headers.
 - `project_id` (`int`) – Identifier of the project to delete.
- **Returns:** The HTTP response from the delete endpoint.
- **Return type:** `requests.Response`

3.0.9 `delete_quota`(`dashboard_base_url`: `str`, `headers`: `dict[str, str]`, `quota_id`: `int`) → `Response`

Delete a quota through the dashboard quotas API.

- **Parameters:**
 - `dashboard_base_url` (`str`) – Base URL of the dashboard backend service.
 - `headers` (`dict [**str*`, `str */`) – Authenticated request headers.
 - `quota_id` (`int`) – Identifier of the quota to delete.
- **Returns:** The HTTP response from the quota delete endpoint.
- **Return type:** `requests.Response`

3.0.10 `delete_user`(`dashboard_base_url`: `str`, `headers`: `dict[str, str]`, `user_id`: `int`) → `Response`

Delete a user through the dashboard users API.

- **Parameters:**

- **dashboard_base_url** (*str*) – Base URL of the dashboard backend service.
- **headers** (*dict* [***str**, *str* **/*]) – Authenticated request headers.
- **user_id** (*int*) – Identifier of the user to delete.

- **Returns:** The HTTP response from the delete endpoint.
- **Return type:** `requests.Response`

3.0.11 find_limit_id_by_name(limit_types: list[dict], limit_name: str) → int

Find a quota limit type ID by its name.

- **Parameters:**
 - **limit_types** (*list* [***dict** **/*]) – List of limit type dictionaries.
 - **limit_name** (*str*) – Limit type name to search for.
- **Returns:** The matching limit type ID.
- **Return type:** *int*

3.0.12 find_user_id_by_email(users: list[dict], email: str) → int

Find a user ID in a user list by email address.

- **Parameters:**
 - **users** (*list* [***dict** **/*]) – List of user dictionaries.
 - **email** (*str*) – Email address to search for.
- **Returns:** The matching user ID.
- **Return type:** *int*

3.0.13 find_user_id_by_username(users: list[dict], username: str) → int

Find a user ID in a user list by username.

- **Parameters:**
 - **users** (*list* [***dict** **/*]) – List of user dictionaries.
 - **username** (*str*) – Username to search for.
- **Returns:** The matching user ID.
- **Return type:** *int*

3.0.14 `get_all_projects(dashboard_base_url: str, headers: dict[str, str]) → list[dict]`

Fetch all projects from the dashboard projects endpoint.

- **Parameters:**
 - `dashboard_base_url` (*str*) – Base URL of the dashboard backend service.
 - `headers` (*dict* [***str**, *str* **/*]) – Authenticated request headers.
- **Returns:** A list of project dictionaries.
- **Return type:** *list[dict]*

3.0.15 `get_limit_types(dashboard_base_url: str, headers: dict[str, str]) → list[dict]`

Fetch available quota limit types from the dashboard API.

- **Parameters:**
 - `dashboard_base_url` (*str*) – Base URL of the dashboard backend service.
 - `headers` (*dict* [***str**, *str* **/*]) – Authenticated request headers.
- **Returns:** A list of quota limit type dictionaries.
- **Return type:** *list[dict]*

3.0.16 `get_project_id_by_name(projects: list[dict], project_name: str) → int`

Find a project ID in a project list by name.

- **Parameters:**
 - `projects` (*list* [***dict** **/*]) – List of project dictionaries.
 - `project_name` (*str*) – Project name to search for.
- **Returns:** The matching project ID.
- **Return type:** *int*

3.0.17 `get_users(dashboard_base_url: str, headers: dict[str, str]) → list[dict]`

Fetch all users from the dashboard users listing endpoint.

- **Parameters:**
 - `dashboard_base_url` (*str*) – Base URL of the dashboard backend service.
 - `headers` (*dict* [***str**, *str* **/*]) – Authenticated request headers.
- **Returns:** A list of user dictionaries.
- **Return type:** *list[dict]*

3.0.18 `load_mapping(completions_dir: Path, mapping_name: str) → dict`

Load a named mock completion mapping from the fixtures directory.

- **Parameters:**
 - **completions_dir** (*Path*) – Directory containing completion mapping JSON files.
 - **mapping_name** (*str*) – File stem of the mapping to load.
- **Returns:** The mapping dictionary, or an empty dictionary if not found.
- **Return type:** *dict*

3.0.19 `login(dashboard_base_url: str, dashboard_request_headers: dict[str, str], email: str, password: str, expected_status: int = 200) → Response`

Authenticate against the dashboard login endpoint and assert expected status.

- **Parameters:**
 - **dashboard_base_url** (*str*) – Base URL of the dashboard backend service.
 - **dashboard_request_headers** (*dict* [***str**, *str* *]) – Required dashboard request headers.
 - **email** (*str*) – Login email address.
 - **password** (*str*) – Login password.
 - **expected_status** (*int*, *optional*) – Expected HTTP status code.
- **Returns:** The HTTP response from the login endpoint.
- **Return type:** `requests.Response`

3.0.20 `post_chat_completion(completions_url: str, api_key: str, request_data: dict) → Response`

Send a chat completion request to the gateway completions endpoint.

- **Parameters:**
 - **completions_url** (*str*) – The base URL prefix for completions requests.
 - **api_key** (*str*) – The API key used as a bearer token.
 - **request_data** (*dict*) – The JSON payload for the completion request.
- **Returns:** The HTTP response returned by the completions endpoint.
- **Return type:** `requests.Response`

3.0.21 `register_user(dashboard_base_url: str, headers: dict[str, str], email: str, username: str, password: str, mandate_reset: bool) → Response`

Register a user through the dashboard authentication API.

- **Parameters:**

- **dashboard_base_url** (*str*) – Base URL of the dashboard backend service.
- **headers** (*dict* [***str**, *str* ***]) – Authenticated request headers.
- **email** (*str*) – New user’s email address.
- **username** (*str*) – New user’s username.
- **password** (*str*) – Initial password for the user.
- **mandate_reset** (*bool*) – Whether the user must reset password on first login.

- **Returns:** The HTTP response from the register endpoint.

- **Return type:** `requests.Response`

3.0.22 `wait_for_health(url: str, timeout: int = 30, interval: float = 1.0, headers: dict[str, str] | None = None) → Response`

Wait until a health endpoint responds with HTTP 200.

- **Parameters:**

- **url** (*str*) – The health-check URL to poll.
- **timeout** (*int* , *optional*) – Maximum wait time in seconds.
- **interval** (*float* , *optional*) – Sleep interval in seconds between retries.
- **headers** (*dict* [***str**, *str* ***] | *None* , *optional*) – Optional HTTP headers sent with each request.

- **Returns:** The successful HTTP response with status code 200.

- **Return type:** `requests.Response`

3.1 Subpackages

- `integration.dashboard` package

- Submodules

- `integration.dashboard.test_auth` module

- * `test_dashboard_login_rejects_wrong_password()`
 - * `test_dashboard_login_returns_bearer_token()`
 - * `test_dashboard_me_returns_admin_user()`

- `integration.dashboard.test_gateway_health` module

- * `test_dashboard_health_endpoint_returns_200()`

- `integration.dashboard.test_keys_and_quotas` module

- * `test_keys_and_quotas_key_access_then_quota_limit_enforced()`

- integration.dashboard.test_projects module
 - * test_projects_add_user_to_project()
 - * test_projects_create_project_then_list_contains_it()
 - * test_projects_delete_project_then_second_delete_fails()
 - * test_projects_remove_user_from_project()
 - * test_projects_seeded_projects_are_listed()
- integration.dashboard.test_users module
 - * test_users_admin_can_change_password_for_user()
 - * test_users_admin_can_delete_user()
 - * test_users_me_shows_new_user_and_password_expired_when_mandated()
 - * test_users_register_invalid_email_returns_verbose_422_detail()
 - * test_users_register_with_mandated_reset()
- integration.gateway package
 - Submodules
 - integration.gateway.test_chat_end_to_end module
 - * test_chat_end_to_end()
 - integration.gateway.test_dashboard_health module
 - * test_health_endpoint_returns_200()
 - integration.gateway.test_key_authentication module
 - * test_key_authentication_rejects_expired_key()
 - * test_key_authentication_rejects_invalid_fingerprint_with_valid_length()
 - * test_key_authentication_rejects_invalid_key_with_matching_fingerprint()
 - * test_key_authentication_rejects_too_long_key()
 - * test_key_authentication_rejects_too_short_key()
 - integration.gateway.test_quota_enforcement module
 - * test_quota_immediate_over_limit_returns_429()
 - * test_quota_within_limit_then_next_request_exceeds_remaining_quota()
 - integration.gateway.test_upstream_error module
 - * test_upstream_error()

3.2 Submodules

3.3 integration.conftest module

3.3.1 `admin_token(dashboard_base_url: str, administrator_email: str, administrator_password: str, dashboard_request_headers: dict[str, str]) → str`

Authenticate once per session and return an admin access token.

3.3.2 `administrator_email() → str`

Return administrator email used for dashboard authentication tests.

3.3.3 administrator_password() → str

Return administrator password used for dashboard authentication tests.

3.3.4 base_url() → str

Return the gateway base URL based on local or CI environment.

3.3.5 completions_dir(fixture_dir: Path) → Path

Return the completions fixture directory containing mock payload mappings.

3.3.6 completions_url(base_url: str) → str

Return the gateway chat-completions API prefix.

3.3.7 dashboard_base_url() → str

Return the dashboard base URL based on local or CI environment.

3.3.8 dashboard_request_headers(frontend_origin: str) → dict[str, str]

Build default request headers expected by dashboard endpoints.

3.3.9 expired_api_key() → str

Return a seeded API key that should be treated as expired or inactive.

3.3.10 fixtures_dir() → Path

Return the root fixtures directory used by integration tests.

3.3.11 frontend_origin() → str

Return the frontend origin value used for CORS-sensitive dashboard calls.

3.3.12 limited_api_key() → str

Return a seeded API key with strict quota limits for quota tests.

3.3.13 sleep_between_tests()

Introduce a short startup delay to reduce race conditions during test bootstrapping.

3.3.14 unlimited_api_key() → str

Return a seeded API key intended to run without quota restrictions.

3.4 integration.helpers module

3.4.1 auth_headers(admin_token: str, dashboard_request_headers: dict[str, str]) → dict[str, str]

Merge dashboard request headers with an admin bearer authorization header.

- **Parameters:**

- **admin_token** (*str*) – Access token for an authenticated administrator.
- **dashboard_request_headers** (*dict* [***str**, *str* **/*]) – Baseline request headers required by dashboard routes.

- **Returns:** A headers dictionary with the Authorization header included.

- **Return type:** *dict*[*str*, *str*]

3.4.2 bearer_headers(token: str) → dict[str, str]

Build Authorization headers for bearer token authentication.

- **Parameters:** **token** (*str*) – The bearer token value.
- **Returns:** A headers dictionary containing the Authorization header.
- **Return type:** *dict*[*str*, *str*]

3.4.3 build_chat_url(completions_url: str) → str

Build the chat completions endpoint URL from a base completions URL.

- **Parameters:** **completions_url** (*str*) – The base URL prefix for completions requests.
- **Returns:** The fully qualified chat completions endpoint URL.
- **Return type:** *str*

3.4.4 create_api_key(dashboard_base_url: str, headers: dict[str, str], project_id: int, name: str) → Response

Create an API key through the dashboard keys API.

- **Parameters:**

- **dashboard_base_url** (*str*) – Base URL of the dashboard backend service.
- **headers** (*dict* [***str**, *str* **/*]) – Authenticated request headers.
- **project_id** (*int*) – Project ID the API key will belong to.
- **name** (*str*) – Human-readable API key label.

- **Returns:** The HTTP response from the create key endpoint.

- **Return type:** *requests.Response*

3.4.5 create_project(dashboard_base_url: str, headers: dict[str, str], project_name: str, manager_id: int) → Response

Create a project through the dashboard projects API.

- **Parameters:**
 - **dashboard_base_url** (*str*) – Base URL of the dashboard backend service.
 - **headers** (*dict* [***str**, *str* **/*]) – Authenticated request headers.
 - **project_name** (*str*) – Name for the new project.
 - **manager_id** (*int*) – User ID assigned as project manager.
- **Returns:** The HTTP response from the create endpoint.
- **Return type:** requests.Response

3.4.6 create_quota(dashboard_base_url: str, headers: dict[str, str], , key_id: int | None = None, project_id: int | None = None, user_id: int | None = None, limit_id: int, limit_value: float, period: str) → Response

Create a quota through the dashboard quotas API.

- **Parameters:**
 - **dashboard_base_url** (*str*) – Base URL of the dashboard backend service.
 - **headers** (*dict* [***str**, *str* **/*]) – Authenticated request headers.
 - **key_id** (*int* | *None* , *optional*) – Optional API key ID target for the quota.
 - **project_id** (*int* | *None* , *optional*) – Optional project ID target for the quota.
 - **user_id** (*int* | *None* , *optional*) – Optional user ID target for the quota.
 - **limit_id** (*int*) – Limit type identifier.
 - **limit_value** (*float*) – Numeric quota amount.
 - **period** (*str*) – Quota period value as API string.
- **Returns:** The HTTP response from the quota create endpoint.
- **Return type:** requests.Response

3.4.7 delete_api_key(dashboard_base_url: str, headers: dict[str, str], key_id: int) → Response

Delete an API key through the dashboard keys API.

- **Parameters:**
 - **dashboard_base_url** (*str*) – Base URL of the dashboard backend service.

- **headers** (*dict* [***str**, *str* ***]) – Authenticated request headers.
- **key_id** (*int*) – Identifier of the API key to delete.
- **Returns:** The HTTP response from the delete key endpoint.
- **Return type:** `requests.Response`

3.4.8 `delete_project(dashboard_base_url: str, headers: dict[str, str], project_id: int) → Response`

Archive or delete a project through the dashboard projects API.

- **Parameters:**
 - **dashboard_base_url** (*str*) – Base URL of the dashboard backend service.
 - **headers** (*dict* [***str**, *str* ***]) – Authenticated request headers.
 - **project_id** (*int*) – Identifier of the project to delete.
- **Returns:** The HTTP response from the delete endpoint.
- **Return type:** `requests.Response`

3.4.9 `delete_quota(dashboard_base_url: str, headers: dict[str, str], quota_id: int) → Response`

Delete a quota through the dashboard quotas API.

- **Parameters:**
 - **dashboard_base_url** (*str*) – Base URL of the dashboard backend service.
 - **headers** (*dict* [***str**, *str* ***]) – Authenticated request headers.
 - **quota_id** (*int*) – Identifier of the quota to delete.
- **Returns:** The HTTP response from the quota delete endpoint.
- **Return type:** `requests.Response`

3.4.10 `delete_user(dashboard_base_url: str, headers: dict[str, str], user_id: int) → Response`

Delete a user through the dashboard users API.

- **Parameters:**
 - **dashboard_base_url** (*str*) – Base URL of the dashboard backend service.
 - **headers** (*dict* [***str**, *str* ***]) – Authenticated request headers.
 - **user_id** (*int*) – Identifier of the user to delete.
- **Returns:** The HTTP response from the delete endpoint.
- **Return type:** `requests.Response`

**3.4.11 find_limit_id_by_name(limit_types: list[dict],
limit_name: str) → int**

Find a quota limit type ID by its name.

- **Parameters:**
 - **limit_types** (*list* [***dict** *]) – List of limit type dictionaries.
 - **limit_name** (*str*) – Limit type name to search for.
- **Returns:** The matching limit type ID.
- **Return type:** *int*

3.4.12 find_user_id_by_email(users: list[dict], email: str) → int

Find a user ID in a user list by email address.

- **Parameters:**
 - **users** (*list* [***dict** *]) – List of user dictionaries.
 - **email** (*str*) – Email address to search for.
- **Returns:** The matching user ID.
- **Return type:** *int*

**3.4.13 find_user_id_by_username(users: list[dict], username: str)
→ int**

Find a user ID in a user list by username.

- **Parameters:**
 - **users** (*list* [***dict** *]) – List of user dictionaries.
 - **username** (*str*) – Username to search for.
- **Returns:** The matching user ID.
- **Return type:** *int*

**3.4.14 get_all_projects/dashboard_base_url: str, headers:
dict[str, str]) → list[dict]**

Fetch all projects from the dashboard projects endpoint.

- **Parameters:**
 - **dashboard_base_url** (*str*) – Base URL of the dashboard backend service.
 - **headers** (*dict* [***str** , *str* *]) – Authenticated request headers.
- **Returns:** A list of project dictionaries.
- **Return type:** *list[dict]*

3.4.15 `get_limit_types(dashboard_base_url: str, headers: dict[str, str]) → list[dict]`

Fetch available quota limit types from the dashboard API.

- **Parameters:**
 - `dashboard_base_url` (*str*) – Base URL of the dashboard backend service.
 - `headers` (*dict* [***str**, *str* **/*]) – Authenticated request headers.
- **Returns:** A list of quota limit type dictionaries.
- **Return type:** *list[dict]*

3.4.16 `get_project_id_by_name(projects: list[dict], project_name: str) → int`

Find a project ID in a project list by name.

- **Parameters:**
 - `projects` (*list* [***dict** **/*]) – List of project dictionaries.
 - `project_name` (*str*) – Project name to search for.
- **Returns:** The matching project ID.
- **Return type:** *int*

3.4.17 `get_users(dashboard_base_url: str, headers: dict[str, str]) → list[dict]`

Fetch all users from the dashboard users listing endpoint.

- **Parameters:**
 - `dashboard_base_url` (*str*) – Base URL of the dashboard backend service.
 - `headers` (*dict* [***str**, *str* **/*]) – Authenticated request headers.
- **Returns:** A list of user dictionaries.
- **Return type:** *list[dict]*

3.4.18 `load_mapping(completions_dir: Path, mapping_name: str) → dict`

Load a named mock completion mapping from the fixtures directory.

- **Parameters:**
 - `completions_dir` (*Path*) – Directory containing completion mapping JSON files.
 - `mapping_name` (*str*) – File stem of the mapping to load.
- **Returns:** The mapping dictionary, or an empty dictionary if not found.
- **Return type:** *dict*

3.4.19 `login(dashboard_base_url: str, dashboard_request_headers: dict[str, str], email: str, password: str, expected_status: int = 200) → Response`

Authenticate against the dashboard login endpoint and assert expected status.

- **Parameters:**

- `dashboard_base_url (str)` – Base URL of the dashboard backend service.
- `dashboard_request_headers (dict [**str* , str *])` – Required dashboard request headers.
- `email (str)` – Login email address.
- `password (str)` – Login password.
- `expected_status (int , optional)` – Expected HTTP status code.

- **Returns:** The HTTP response from the login endpoint.

- **Return type:** `requests.Response`

3.4.20 `post_chat_completion(completions_url: str, api_key: str, request_data: dict) → Response`

Send a chat completion request to the gateway completions endpoint.

- **Parameters:**

- `completions_url (str)` – The base URL prefix for completions requests.
- `api_key (str)` – The API key used as a bearer token.
- `request_data (dict)` – The JSON payload for the completion request.

- **Returns:** The HTTP response returned by the completions endpoint.

- **Return type:** `requests.Response`

3.4.21 `register_user(dashboard_base_url: str, headers: dict[str, str], email: str, username: str, password: str, mandate_reset: bool) → Response`

Register a user through the dashboard authentication API.

- **Parameters:**

- `dashboard_base_url (str)` – Base URL of the dashboard backend service.
- `headers (dict [**str* , str *])` – Authenticated request headers.
- `email (str)` – New user's email address.
- `username (str)` – New user's username.
- `password (str)` – Initial password for the user.

- **mandate_reset** (*bool*) – Whether the user must reset password on first login.

- **Returns:** The HTTP response from the register endpoint.
- **Return type:** requests.Response

3.4.22 wait_for_health(url: str, timeout: int = 30, interval: float = 1.0, headers: dict[str, str] | None = None) → Response

Wait until a health endpoint responds with HTTP 200.

- **Parameters:**
 - **url** (*str*) – The health-check URL to poll.
 - **timeout** (*int* , *optional*) – Maximum wait time in seconds.
 - **interval** (*float* , *optional*) – Sleep interval in seconds between retries.
 - **headers** (*dict* [***str** , *str* ***] / *None* , *optional*) – Optional HTTP headers sent with each request.
- **Returns:** The successful HTTP response with status code 200.
- **Return type:** requests.Response

4 integration

- integration package
 - auth_headers()
 - bearer_headers()
 - build_chat_url()
 - create_api_key()
 - create_project()
 - create_quota()
 - delete_api_key()
 - delete_project()
 - delete_quota()
 - delete_user()
 - find_limit_id_by_name()
 - find_user_id_by_email()
 - find_user_id_by_username()
 - get_all_projects()
 - get_limit_types()
 - get_project_id_by_name()
 - get_users()
 - load_mapping()
 - login()
 - post_chat_completion()
 - register_user()
 - wait_for_health()

- Subpackages
 - * integration.dashboard package
 - Submodules
 - integration.dashboard.test_auth module
 - integration.dashboard.test_gateway_health module
 - integration.dashboard.test_keys_and_quotas module
 - integration.dashboard.test_projects module
 - integration.dashboard.test_users module
 - * integration.gateway package
 - Submodules
 - integration.gateway.test_chat_end_to_end module
 - integration.gateway.test_dashboard_health module
 - integration.gateway.test_key_authentication module
 - integration.gateway.test_quota_enforcement module
 - integration.gateway.test_upstream_error module
- Submodules
- integration.conftest module
 - * admin_token()
 - * administrator_email()
 - * administrator_password()
 - * base_url()
 - * completions_dir()
 - * completions_url()
 - * dashboard_base_url()
 - * dashboard_request_headers()
 - * expired_api_key()
 - * fixtures_dir()
 - * frontend_origin()
 - * limited_api_key()
 - * sleep_between_tests()
 - * unlimited_api_key()
- integration.helpers module
 - * auth_headers()
 - * bearer_headers()
 - * build_chat_url()
 - * create_api_key()
 - * create_project()
 - * create_quota()
 - * delete_api_key()
 - * delete_project()
 - * delete_quota()
 - * delete_user()
 - * find_limit_id_by_name()
 - * find_user_id_by_email()

- * find_user_id_by_username()
- * get_all_projects()
- * get_limit_types()
- * get_project_id_by_name()
- * get_users()
- * load_mapping()
- * login()
- * post_chat_completion()
- * register_user()
- * wait_for_health()